ITF23019: Parallel and Distributed Programming

Report for Lab 11: Socket Programming

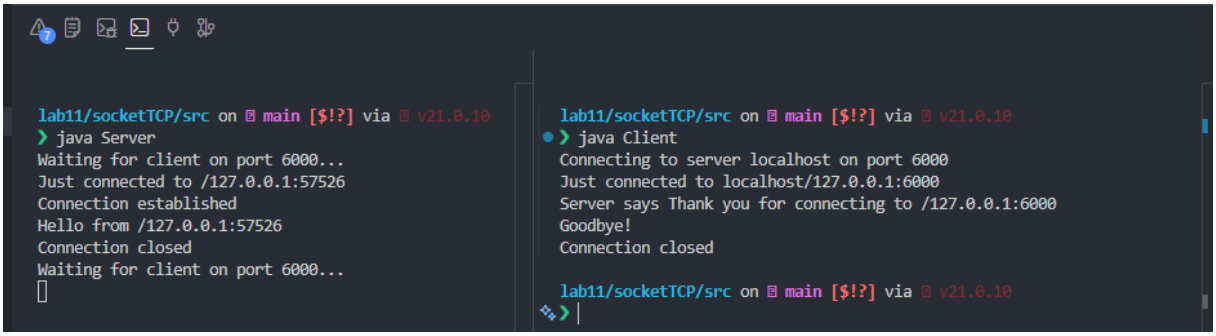
Name: Emil Berglund

GitHub: EmilB04

Exercise 1: SocketTCP

Output:

- **Subtask 1: Run code, include output:**



- **Subtask 2: Which class should run first?**
  - The Server should always run first. The reason for this is because of Socket. Client immediately tries to establish a connection when it runs. This means, if the Server is not running, the connection will fail.
- **Subtask 3: Which port number is used for Client, and which for Server?**
  - Server: Listens on port: 6000
  - Client: Assigned port: 67526
- **Subtask 4: What is the IP address or hostname of Client and Server?**
  - Both utilize localhost but connect via different ports:
    - Server is at 127.0.0.1:6000
    - Client is at 127.0.0.1:57526

Server:

- **Subtask 5: Which line the Server opens the socket?**
  - The server opens the socket on this line in the constructor (line 8): *serverSocket = new ServerSocket(port);*
- **Subtask 6: Which line the Server waits for Client?**
  - The server waits for a client on this line (line 17): $Socket\ server\ =\ serverSocket.accept();$
- **Subtask 7: Which lines the Server creates I/O Streams for communication with the Client?**
  - The server creates I/O streams on these two lines (lines 21 & 24):

$$DataInputStream\ in\ =$$
$$new\ DataInputStream(server.getInputStream());$$
$$DataOutputStream\ out\ =$$
$$new\ DataOutputStream(server.getOutputStream());$$

- **Subtask 8: Which lines the Server performs communication with the Client?**
  - The server performs communication on these two lines (lines 23 & 26):

$System.out.println(in.readUTF());$

$out.writeUTF("Thank\ you\ for\ connecting\ to\ " +$

$server.getLocalSocketAddress() + "\nGoodbye!");$

- **Subtask 9: Which line the connection is closed?**
  - The connection is closed at line 27 with the following command:

    server.close();

Client:

- **Subtask 10: Which line the Client creates a socket object?**
  - The Client creates a socket object at line 11:

    $Socket\ client\ =\ new\ Socket(serverName, port);$

- **Subtask 11: Which lines the Client creates I/O Streams for communication with the Cilent?**
  - Client creates I/O Streams at lines 14 and 15 (out), and 18 and 19 (in):

    $OutputStream\ outToServer\ =\ client.getOutputStream(\ );$

    $DataOutputStream\ out\ =\ new\ DataOutputStream(outToServer);$

    $InputStream\ inFromServer\ =\ client.getInputStream();$

    $DataInputStream\ in\ =\ new\ DataInputStream(inFromServer);$

- **Subtask 12: Which lines the Client performs communication with the Client?**
  - Client performs communication at line 17 and 21:

    $out.writeUTF("Hello\ from\ " + client.getLocalSocketAddress());$

    $System.out.println("Server\ says\ " + in.readUTF());$

- **Subtask 13: Which line the connection is closed?**
  - The Client closed the connection at line 22:

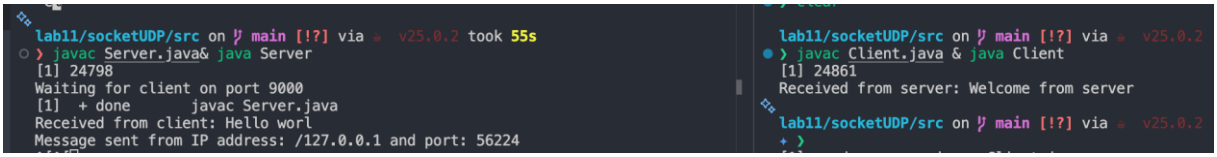
    $client.close();$

Finally:

- **Subtask 14: How do you relate the above operations of Client and Server to the Three-way handshake data transmission in TCP protocol?**
  - Not answered

Exercise 2: SocketUDP

Subtask 1: Run the code and include output:

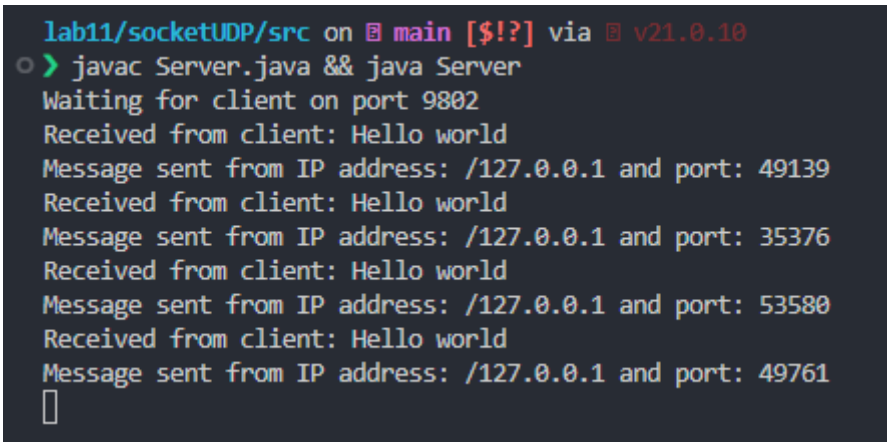


Subtask 2: Which class should run first?

- This is the same situation as in Exercise 1. The Server should always run first. The reason for this is because of Socket. Client immediately tries to establish a connection when it runs. This means, if the Server is not running, the connection will fail.

Subtask 3: Create the output similar to the figure below. How and why can the server receive multiple requests with multiple ports from client(s)?

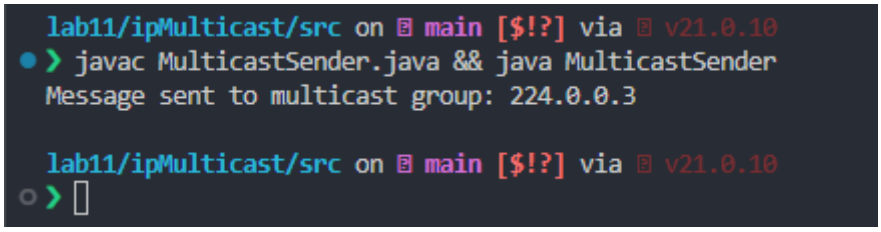
- To recreate the output to the provided figure, I had to do the following
    1. Change server-port to 9802 from 9000
    2. Change receivingData byte size from 10 to 11 (Hello world was cut)
- The server can receive multiple requests with multiple port from client(s) because:
    - UDP is connectionless with one server socket which can handle unlimited clients. Each client is independent.
    - Each client gets a port from the OS when Socket is created, so there is no port conflict.



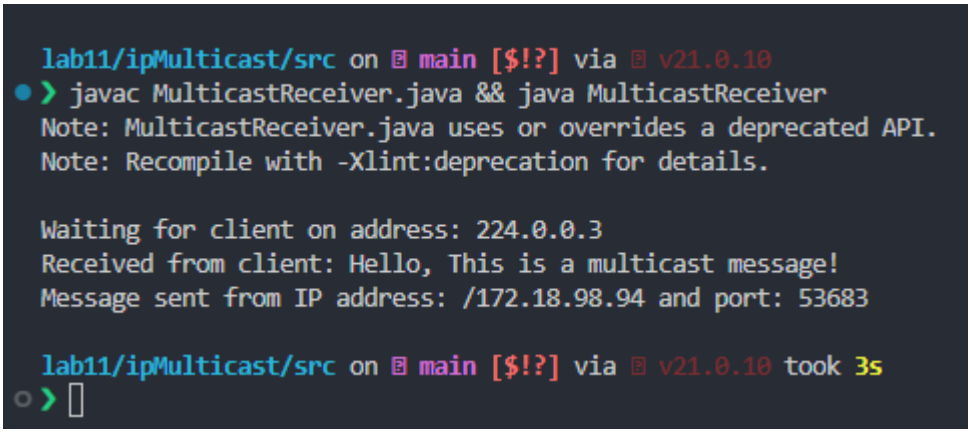
Exercise 3: Multicast

Subtask 1: Reproduce Figures 6 and 7

- Figure 6: Code changes are pushed



- Figure 7: Code changes are pushed



Subtask 2: Can we set up multiple senders in the same way with setting multiple receivers? Explain why and why not.

- Yes. Multicast supports many senders and many receivers on the same group address and port, but not always.
    - Why:
        - Senders send to one multicast group
        - Any receivers that joined the group gets the packets
    - Why not always:
        - Network/router may block multicast
        - UDP gives no guaranteed delivery/order
        - Same-host multiple receivers may need socket reuse.